把本项目部署到自己的 GitHub + Vercel

详细图文版 · 适用对象：第一次部署别人的项目，或想在自己电脑/服务器上跑一份

预计耗时：纯部署 40–60 分钟；想在本地改代码再加 30 分钟

你需要：GitHub 账号、Vercel 账号（用 GitHub 登录即可）、一个 OpenAI 兼容的大模型 Key（如 DeepSeek）

本指南的约定：带 ⚠ 的都是**真实踩过的坑**，不是理论风险；带「预期输出」的都可对照检查。

目录

1. 这份指南怎么用
2. 先理解这个项目是什么形态
3. 准备工作（账号、软件、密钥）
4. 第一步：拿到代码
5. 第二步：上传到自己的 GitHub
6. 第三步：在 Vercel 导入并部署
7. 第四步：部署后的 5 条验证
8. 第五步（可选）：绑定自己的域名
9. 第六步（可选）：部署到自己的云服务器
10. 三种部署位置怎么选
11. 日常维护：改代码、回滚、看日志
12. 常见问题（15 条，带报错原文）
13. 安全清单
14. 命令速查（每条都解释）
15. 附录：目录结构与你不能动的文件

1 这份指南怎么用

先按你的目标选路线，不用全部读完：

你的目标	读哪几节	时间
只是想看看项目长什么样	第 2 节 + 打开演示地址	5 分钟
自己部署一份（推荐）	1 → 2 → 3 → 4 → 5 → 6 → 7	40–60 分钟
部署在自己的域名上	上面 + 第 8 节	+20 分钟
部署在自己的云服务器上	1 → 2 → 3 → 4 → 9	+40 分钟

1.1 先认识 8 个词（后面反复出现）

词	一句话解释
仓库（Repository）	GitHub 上的一个项目文件夹，带完整修改历史
Fork	把别人的仓库复制一份到你账号下，之后你随便改，不影响原作者
clone / push / pull	把仓库下载到本地 / 把本地修改传上去 / 把别人的修改拉下来
commit	一次「保存点」，带一句说明，历史里可以回退
分支（branch）	一条独立的修改线；`main` 是主分支，部署平台默认盯着它
环境变量（Environment Variables）	存在服务器上的配置（密钥这类），**不写进代码**，改完要重新部署才生效
部署（Deploy）	让平台拿你的代码跑起来，产出一个可访问的网址
构建（Build）	部署过程中的「编译」阶段；本项目在构建时会额外下载模型文件

2 先理解这个项目是什么形态

不先搞懂这一点，后面看到「下载 23 MB 模型」「访问很慢」会以为是部署失败。

2.1 计算几乎都在浏览器里

组成	跑在哪	说明
页面与交互（Next.js 16）	浏览器	静态路由，Vercel 只负责发 HTML/JS
文件解析、质量检查、清洗、导出	**浏览器**（含 Web Worker）	用户上传的文件**不会离开他的电脑**
端侧语义模型（23 MB）+ 推理运行时（53 MB）	**浏览器**（WebGPU 优先，退 WASM）	由站点自己托管，不依赖第三方 CDN
大模型调用	服务器上的一个转发接口 `/api/agent`	只转发「结构摘要」（列名、缺失率这类），不碰原始数据

2.2 这对部署意味着什么

1. **服务器很轻**：没有数据库、不存文件、不存会话，Vercel 免费版基本够用；

2. **首次打开页面要下 76 MB**（模型 + 推理运行时），所以「第一次慢、第二次快」是正常的；

3. **这 76 MB 不在 Git 仓库里**（.gitignore 排除了 public/models/、public/ort/），

它们是**构建时下载/拷贝**出来的 —— 这一点是部署失败最常见的根因，第 6.6 节细说；

4. **AI 分析必须有 Key**，没有 Key 其余功能都能用，只有「AI 分析」会报错。

3 准备工作

3.1 三个账号

账号	怎么弄
GitHub	github.com 注册，需要邮箱验证
Vercel	vercel.com 点 **Continue with GitHub**，授权一下就用 GitHub 账号登录了（不用再注册）
大模型服务	例如 platform.deepseek.com 注册 → 充值少量余额（够测试）→ 创建 API Key

3.2 一个 Key（后面要填三次）

在 DeepSeek 控制台 → **API keys** → **创建 API key** → 复制那串 sk-...（**只显示一次**，先存到记事本）。

同时记住两个东西（后面填环境变量要用）：

- 接口地址：`https://api.deepseek.com`
- 模型名：`deepseek-chat`

如果你用的是别家（OpenAI 兼容接口都行），把这两个换成对应值即可。

3.3 本地软件（只有想在自己电脑上跑/改代码才需要）

软件	检查命令	预期输出
Git	`git --version`	`git version 2.x`
Node	`node -v`	`v24.x`（本项目要求 ≥ 24）
pnpm	`pnpm -v`	`10.x` 或以上

装 pnpm（Node 装好后）：`npm i -g pnpm`

*纯在 Vercel 上部署**不需要**本地装 Node/pnpm，只需要 Git。Vercel 的构建机自带环境。*

4 第一步：拿到代码

4.1 方式 A：Fork（推荐）

1. 打开原仓库页面：`https://github.com/HUA-T-SVG/AI-web`

2. 右上角找到 **Fork**（一个分叉图标），点它

3. 下一页点 **Create fork**，等几秒

4. 完成后地址变成 `https://github.com/你的用户名/AI-web` —— 这就是你的仓库了

为什么推荐 Fork：作者以后更新了代码，你可以在自己仓库点 **Sync fork** 一键同步，不用重新下 ZIP。

4.2 方式 B：下载 ZIP 自己建仓库

1. 仓库页 → 绿色 **Code** 按钮 → **Download ZIP**

2. 解压到**路径里不要有中文和空格**的目录，例如 `D:\projects\ai-web`

⚠ 路径带中文（比如 `D:\我的项目\ai-web`）会让某些构建工具报奇怪的错，别省这一步

3. 进目录，初始化 Git：

```
cd D:\projects\ai-web
git init
git add .
git commit -m "init: 导入项目代码"
```

4.3 验证你拿到了完整代码

目录里应该能看到这些（**缺一个都说明没下全**）：

```
ai-web/
├─ src/                 # 源码（core / parser / embedding / agent / ui / knowledge / scriptgen）
├─ scripts/             # 构建脚本：fetch-model.mjs、sync-ort-wasm.mjs 等
├─ knowledge/           # 规则库与评测产物
├─ tests/               # 单元测试与 E2E
├─ public/              # 静态资源（models/ 与 ort/ 是构建时生成的，现在没有是正常的）
├─ package.json
├─ next.config.ts
├─ .node-version        # 写死 Node 24，Vercel 会读它
└─ .env.example         # 环境变量模板（真正的 .env.local 不该存在仓库里）
```

5 第二步：上传到自己的 GitHub

5.1 新建一个空仓库

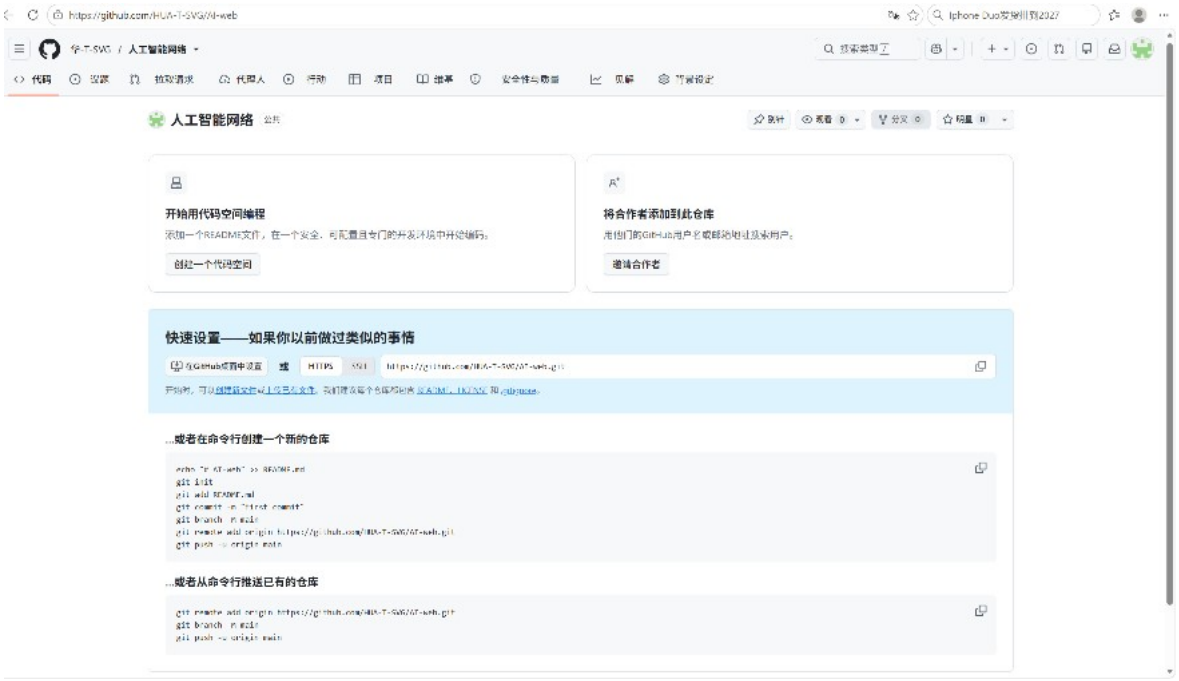
1. GitHub 右上角 **+** → **New repository**

2. 填写：

- **Repository name**：`ai-web`（随便起，建议不带中文）
- **Description**：可留空
- **Public / Private**：Public 可以省 Vercel 授权麻烦，Private 也可以
- ⚠ **下面三个勾选框全部不勾**（Add a README / .gitignore / license）

3. 点 **Create repository**

创建成功后页面会显示「快速设置」，里面直接给了推送命令 —— 下图就是真实界面：



（图 1：新建空仓库后页面给出的推送命令（线上真实界面））

5.2 推送代码（对应图 1 里的命令）

如果你是方式 A（Fork）：仓库里已经有代码，**跳过本节**，直接去第 6 节。

如果你是方式 B（ZIP），在项目目录执行：

```
cd D:\projects\ai-web
git remote add origin https://github.com/你的用户名/ai-web.git
git branch -M main
git push -u origin main
```

每条命令在做什么：

命令	作用	关键提示
`git remote add origin ...`	告诉本地「远程仓库在哪」	地址写错会报 `repository not found`
`git branch -M main`	把当前分支改名成 `main`	Vercel 默认只认 `main`
`git push -u origin main`	第一次推送，并把 `main` 设为默认上游	第一次会要求登录

预期输出（正常时）：

```
Enumerating objects: 412, done.
...
To https://github.com/你的用户名/ai-web.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

5.3 ⚠ 第一次推送要求登录：选「浏览器登录」

Windows 会弹出一个 GitHub 登录窗口（Git Credential Manager）：

- 选 **Sign in with your browser**（浏览器登录）
- 浏览器里点 **Authorize**
- 回到命令行，推送继续

如果命令行卡在 `Username for 'https://github.com':` 而**没有弹窗**：按 `Ctrl+C` 中断，执行一次下面这行再重试：

```
git config --global credential.helper manager
```

凭证会记住，下次不用再登录。

5.4 ⚠ 别把密钥推上去

推送前检查：

```
git ls-files | findstr env
```

正确输出：只有 `.env.example`

错误输出：出现 `.env.local` → 说明被跟踪了，执行：

```
git rm --cached .env.local
git commit -m "chore: 不再跟踪本地环境变量"
```

`.gitignore` 里已有 `*.env.local`，**不要删掉这一行**。

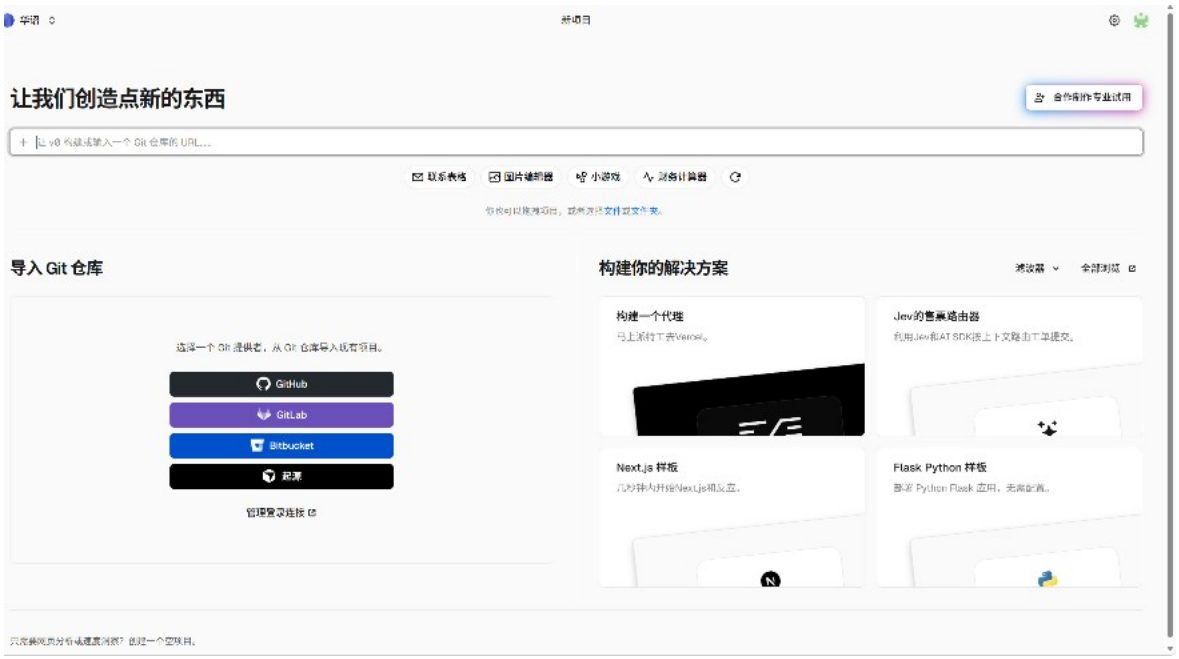
5.5 推送后验证

刷新 GitHub 页面，应该能看到 `src/`、`scripts/`、`package.json` 等文件，以及提交记录里有你刚才的 commit。

6 第三步：在 Vercel 导入并部署

6.1 登录并新建项目

1. 打开 vercel.com → **Continue with GitHub**
2. 进入控制台后点 **Add New… → Project**
3. 看到「导入 Git 仓库」，点 **GitHub**（第一次会要求授权 Vercel 访问你的仓库，选 **All repositories** 或只勾这一个）



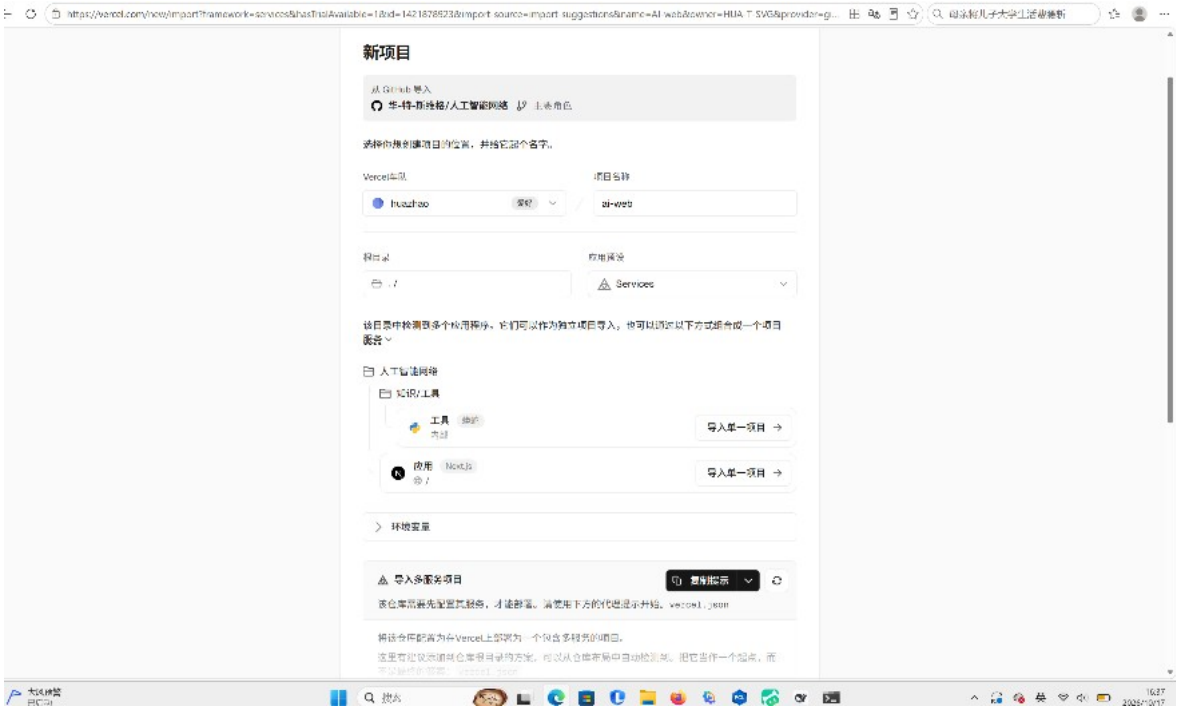
（图 2：Vercel 新项目页，选择 GitHub 导入（线上真实界面））

6.2 ⚠ 关键一步：它可能说「检测到多个应用程序」

这个仓库里除了主应用还有别的目录（知识库工具等），所以 Vercel 可能把它识别成「多服务项目」，然后要求你配置 `vercel.json`。

不要走那条路。正确做法是在检测结果列表里：

- 找到 **「应用 · Next.js」** 那一行
- 点它右边的 **「导入单一项目」**



（图 3：Vercel 项目配置页，中间的「导入单一项目」才是正确入口（线上真实界面））

选对之后，配置页保持默认即可：

字段	填什么	说明
Project Name	任意，如 `ai-web`	会变成 `ai-web.vercel.app`
Framework Preset	**Next.js**（通常已自动识别）	没识别就手动选
Root Directory	`./`	**别改**，改了会找不到 `package.json`
Build Command	留空（默认 `pnpm build`）	别填成 `next build`，会跳过 prebuild
Output Directory	留空	Next.js 自动处理
Install Command	留空（默认 `pnpm install`）	别改

6.3 填三个环境变量（必填，但只影响 AI 分析）

在 Environment Variables 区域加三条：

变量名	填什么	例子
`LLM_API_KEY`	你的 Key	`sk-xxxxxxxx`
`LLM_BASE_URL`	接口地址	`https://api.deepseek.com`
`LLM_MODEL`	模型名	`deepseek-chat`

三条都要选 **Production / Preview / Development 全勾**（或者直接选 All Environments）。

⚠ **三个常见错误：**

1. 变量名写成 `NEXT_PUBLIC_LLM_API_KEY` —— 会让 Key 被打进浏览器 JS，**任何人按 F12 都能看到**；
2. `LLM_BASE_URL` 末尾加了 `/v1`（DeepSeek 不需要，OpenAI 需要）—— 填错会报 404；
3. 变量名拼错（`LLM_BASEURL`、`LLM_KEY`）—— 页面不报错，只是 AI 分析一直失败。

截图给别人看时，用输入框右边的「敏感值」开关（显示成点），或直接打码。

6.4 点 Deploy

点按钮后进入构建日志页面。**这一步会持续 3–6 分钟**，因为本项目构建时要下载两个资源（见 6.6）。

6.5 怎么看构建是否成功

页面顶部状态：

状态	含义	你要做什么
Building	正在构建	等
Ready	成功，已上线	点 **Visit** 打开
Error	构建失败	往下拉日志，看第一行红色报错

日志里正常应该**能看到**这三段（按顺序）：

```
> node scripts/fetch-model.mjs        ← 下载端侧模型
> node scripts/sync-ort-wasm.mjs      ← 拷贝推理运行时
> next build                           ← 编译
```

6.6 ⚠ 这 76 MB 是构建时下载的，不在仓库里

构建阶段	做什么	打到哪	耗时
prebuild → scripts/fetch-model.mjs	下载 bge-small-zh-v1.5 模型	public/models/	1–3 分钟
prebuild → scripts/sync-ort-wasm.mjs	拷贝 onnxruntime-web 的 WASM	public/ort/	10–30 秒
next build	编译 Next.js	.next/	1–2 分钟

这两个目录**被 .gitignore 排除了**，所以：

- 仓库里看不到它们是**正常的**；
- 如果构建日志里**没有**这两段 → 说明 package.json 的 prebuild 被改过，恢复它；
- 如果日志里有但报网络错误 → 构建机连不上模型源，重试或改用方式 B（本地先生成再提交，见 12.3）。

7 第四步：部署后的 5 条验证

Vercel 给你一个 https://xxx.vercel.app 地址。**先别急着发给别人**，按顺序点一遍：

#	操作	预期结果	不符合时看
1	打开首页	出现「把数据文件拖到这里」的上传区	打不开 = 构建失败，回 Vercel 看日志
2	上传一个 CSV/Excel	出现预览表格，列名对得上	报错 = 文件编码/格式问题（换 UTF-8 CSV 试）
3	点「质量检查」	出现列画像 + 问题清单	一直空白 = 检查引擎异常，看浏览器 F12 Console
4	看顶部状态条	十几秒后变「**语义归并已就绪，本次检查包含 L3 语义层**」	见 12.4
5	点「AI 分析」问一句	有流式回答	报「接口地址或密钥无效」= 环境变量问题，见 12.5

7.1 第 1–2 步看到的界面



（图 4：正常部署后，首页上传区与预览表格（线上真实界面））

7.2 第 4 步为什么最重要

它判断的是「端侧模型能不能在你的浏览器里加载」。这一步过了，说明：

- 构建时下载的模型与 WASM 真的被打进了部署产物；
- 浏览器能同源加载它们（没有跨域问题）。



（图 5：状态条显示「语义归并已就绪」，且列画像与问题清单都在（线上真实界面））

如果这一步不过，其余功能（确定性检查、清洗、导出）仍然可用 —— 项目设计了降级，界面会明确写「本次没有被检查」，不会骗你。

8 第五步（可选）：绑定自己的域名

`*.vercel.app` 在境内常打不开（这不是你的项目坏了，是域名被阻断）。绑自己的域名就能解决。

8.1 在 Vercel 加域名

1. 项目 → Settings → Domains
2. 输入 ai.你的域名.com → Add
3. Vercel 给出两种记录之一：A 记录（指向 76.76.21.21）或 CNAME（指向 cname.vercel-dns.com）

8.2 在域名服务商加解析

以阿里云为例：控制台 → **域名** → 点你的域名 → **解析设置** → **添加记录**：

记录类型	主机记录	记录值	TTL
CNAME	ai	cname.vercel-dns.com	600

腾讯云/Cloudflare 同理，只是界面位置不同。

8.3 验证

等 5–30 分钟，然后在自己的电脑上：

```
nslookup ai.你的域名.com
```

能解析出 Vercel 的地址就说明生效了；再打开 https://ai.你的域名.com，Vercel 会自动签发 HTTPS 证书（第一次可能等 1–2 分钟）。

⚠ **一个域名不能同时指向两个平台**。如果域名已经指向别处，只能用子域名（ai.、app. 之类）。

9 第六步（可选）：部署到自己的云服务器

不想用 Vercel、而是有一台阿里云/腾讯云 Linux 服务器时，可以自建（本项目只需要 Node 常驻进程）。

9.1 装环境

```
# Ubuntu 22.04 为例
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs git nginx
sudo npm i -g pnpm
node -v      # 期望 v24.x
```

9.2 拉代码并构建

```
sudo mkdir -p /opt && cd /opt
sudo git clone https://github.com/你的用户名/ai-web.git
cd ai-web
sudo pnpm install
```

写环境变量（**只在服务器上，不进 git**）：

```
sudo tee .env.local > /dev/null <<'EOF'
LLM_API_KEY=sk-你的key
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat
EOF
```

构建（**这一步会下载 76 MB，服务器要能访问外网**）：

```
sudo pnpm build
```

9.3 用 systemd 常驻

新建 /etc/systemd/system/ai-web.service：

```
[Unit]
Description=AI web data quality agent
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/ai-web
ExecStart=/usr/bin/pnpm start
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

启动并开机自启：

```
sudo systemctl daemon-reload
sudo systemctl enable --now ai-web
sudo systemctl status ai-web      # 看是否 active (running)
sudo journalctl -u ai-web -f      # 实时看日志
```

9.4 Nginx 反代 + HTTPS

```
server {
    listen 80;
    server_name ai.example.com;
    client_max_body_size 60m;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 300s;
    }
}
```

```
sudo nginx -t && sudo systemctl reload nginx
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d ai.example.com
```

9.5 ⚠ 自建时最容易踩的三个点

现象	原因	解决
AI 回答到一半停住	Nginx 默认 `proxy_read_timeout 60s` 掐断了 SSE 流	调成 `300s`（见上面配置）
页面一直「语义归并准备中」	`pnpm build` 没跑完，`public/models/` 不存在	重新 `pnpm build`，看日志有没有 `fetch-model.mjs`
构建卡住不动	服务器访问不了 GitHub / 模型源	配代理，或本地构建后把 `public/models`、`public/ort` 传上去

10 三种部署位置怎么选

方案	谁来做	优点	代价
A. 自己的 Vercel（推荐）	朋友按第 6 节	免费、零运维、自动 HTTPS	地址是 *.vercel.app，境内可能打不开；要绑自己域名
B. 借别人的 Vercel 账号	仓库 Transfer 或授权给账号持有者	不用自己弄域名	**共用同一个 Key 额度**；构建次数也算进同一个额度；互相 push 会互相触发部署
C. 自己的云服务器	按第 9 节	完全可控、可绑任意域名	要自己维护；服务器要能出网下载 76 MB；带宽成本

10.1 如果要用「别人的 Vercel 账号」

1. 朋友先按第 4–5 节把自己的仓库准备好；
2. 账号持有者在 Vercel 里 **Add New → Project**，导入**朋友的仓库**；
3. 项目名区分开（如 `ai-web-friend`），**不要复用原项目**；
4. 环境变量：可以用自己的 Key（额度共用），也可以让朋友填自己的；
5. 域名加子域名 `friend.你的域名.com` → DNS 加 CNAME。

⚠ **绝对不要**两个人往同一个 Vercel 项目反复 push：会互相覆盖，出问题无法定位。

11 日常维护

11.1 改代码并上线

```
git add .
git commit -m "说明这次改了什么"
git push
```

推到 `main` 后，Vercel 会**自动**开始新部署（约 2–6 分钟）。在 Project → **Deployments** 能看到进度。

11.2 改了环境变量

⚠ **不会自动生效**：Settings 里改完后 → **Deployments** → 最新那条 → 右侧 ⋯ → **Redeploy**。

11.3 回滚到上一个版本

Deployments → 找到上一个 **Ready** 的部署 → ⋯ → **Promote to Production**（或 **Rollback**）。

11.4 看运行日志

Project → **Logs**（或 Deployments → 某个部署 → **Functions** → 选 `/api/agent`）。AI 分析报错时，这里能看到真实原因。

11.5 删除项目

Project → Settings → 底部 **Delete Project**。注意：绑过的域名要先解绑。

12 常见问题（15 条）

12.1 打不开 xxx.vercel.app

现象：浏览器一直转圈或「连接超时」。

原因：该域名在境内被阻断，不是项目坏了。

解决：绑自己的域名（第 8 节）。这是必须买域名唯一的原因。

12.2 构建失败：Command "pnpm install" exited with 1

先看：日志里 `ERR_PNPM_...` 后面那句话。

日志片段	原因	解决
`ERR_PNPM_NO_LOCKFILE`	`pnpm-lock.yaml` 没提交	`git add pnpm-lock.yaml` 后重推
`ERR_PNPM_OUTDATED_LOCKFILE`	改过 `package.json` 没更新锁文件	本地 `pnpm install` 后提交锁文件
`Unsupported engine`	Node 版本不符	确认 `.node-version` 存在且内容为 `24`

12.3 构建日志里没有 fetch-model.mjs

原因：`package.json` 的 `prebuild` 被改掉了。

解决：确认 `"prebuild": "node scripts/fetch-model.mjs && node scripts/sync-ort-wasm.mjs"` 还在；或者本地跑一遍 `pnpm build`，把生成的 `public/models`、`public/ort` 一起提交（体积大但能跑）。

12.4 状态条写「语义归并不可用」

这是设计好的降级，不是崩溃：确定性检查照常跑，界面会写明「本次没有被检查」。

可能原因	怎么确认
浏览器不支持 WebGPU，且 WASM 也被禁用	换 Chrome/Edge 最新版
`public/models/` 或 `public/ort/` 资源 404	F12 → Network 看这两个目录的请求
部署产物不完整（构建被截断）	看构建日志是否跑到 `next build` 结束

12.5 AI 分析报「接口地址或密钥无效」

1. 变量名是否拼对（`LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL`）；
2. `LLM_BASE_URL` 是否需要 `/v1`（DeepSeek 不要，OpenAI 要）；
3. Key 是否还有余额；

4. 改完变量**是否 Redeploy**（不改不生效）。

12.6 页面 500 / MIDDLEWARE_INVOCATION_FAILED

多数是 Node 版本问题：确认 `.node-version` 是 `24`，且没在 `package.json` 里写死别的 `engines`。

12.7 上传文件报「文件过大」

前端限制 50 MB、20 万行。换小一点的文件，或先按行拆分。

12.8 git push 超时 / Failed to connect to github.com port 443

网络问题，不是配置问题。重试 3–6 次通常就过；仍不行换网络或开代理，**不要去改 git 配置**。

12.9 git push 报 rejected (fetch first)

远端有你本地没有的提交（常见于在 GitHub 网页上建仓库时勾了 README）：

```
git pull --rebase origin main
git push
```

12.10 404 / This page could not be found

Root Directory 被改过，或者 Build Command 被填成 `next build`（跳过 prebuild）。恢复默认后再部署。

12.11 部署成功但样式全丢

浏览器缓存 / CDN 缓存。Ctrl+F5 强刷；仍不对就 Redeploy 一次。

12.12 改完代码线上没变化

- 推的是不是 `main`?
- 最新那次部署状态是不是 **Ready**（不是 Error / 被 Cancel）?
- 是不是在看旧的部署地址（Preview URL 与 Production URL 不同）?

12.13 两个人同时开发互相覆盖

别共用项目。一个人 fork 一个仓库、一个 Vercel 项目。

12.14 想加密码保护（不让别人随便用）

Vercel 免费版没有内置密码保护。可行做法：在 `/api/agent` 里加一个简单的 `Authorization` 校验 + 前端带上固定口令；或者只把地址发给少数人。

12.15 费用会不会超

Vercel Hobby 免费额度足够个人演示；真正花钱的是**大模型调用**（本项目单次分析约几分钱）。公开站点没有鉴权，别人也能消耗你的额度 —— 见第 13 节。

13 安全清单

事项	做法
Key 只放服务端	变量名**不要**加 NEXT_PUBLIC_
别提交 .env.local	.gitignore 已覆盖；不要 git add -f
截图遮 Key	用「敏感值」开关或打码
公开地址会烧额度	本项目接口无鉴权限流；长期公开建议加限流或口令
不要共享同一个人 Key 给多人	一人一个 Key，方便排查与限额
域名与证书	Vercel 自动 HTTPS；自建用 certbot，别用自签证书

14 命令速查（附解释）

```
# ── 本地运行（可选）──
pnpm install          # 装依赖
pnpm dev              # 本地开发服务器 http://localhost:3000
pnpm build            # 生产构建（会下载模型；本地跑一次能提前发现问题）
pnpm start            # 以生产方式启动

# ── Git ──
git init                                # 目录变成 git 仓库
git add .                               # 暂存所有改动
git commit -m "说明"                     # 打一个保存点
git remote add origin <仓库地址>         # 绑定远程仓库
git branch -M main                      # 当前分支改名 main
git push -u origin main                 # 第一次推送并设置上游
git push                                # 之后每次推送
git pull --rebase origin main           # 远端有新提交时同步
git status                              # 看当前改了什么
git log --oneline -5                    # 看最近 5 个提交

# ── 排查 ──
git ls-files | findstr env              # 检查有没有把 .env 提交上去
node -v ; pnpm -v                       # 版本自检
```

15 附录：目录结构与你不能动的文件

15.1 关键目录

路径	作用
src/core/	纯 TS 的检查/清洗/脚本生成引擎（不依赖浏览器或 Node，可复用）
src/parser/	CSV/Excel 解析与编码识别（跑在 Worker 里）
src/embedding/	端侧语义模型加载
src/agent/	大模型接口与浏览器端工具
src/knowledge/	规则库检索（RAG）
src/ui/	界面组件与状态
scripts/fetch-model.mjs	**构建时**下载模型
scripts/sync-ort-wasm.mjs	**构建时**拷贝推理运行时

路径	作用
public/models/ public/ort/	上面两个脚本的产物，**不在 git 里**

15.2 ⚠ 不要乱动的文件

文件	为什么
.node-version	写死 Node 版本，删了可能构建失败
package.json 里的 prebuild / predev	删了线上会缺模型，页面一直「准备中」
.gitignore 里的 public/models/ public/ort/	不忽略会让仓库膨胀 76 MB，且每次构建重复提交
next.config.ts	里面有 WASM 与模型加载的相关配置

15.3 部署完成后你想改什么，改哪里

想改	改哪
页面文案	src/ui/ 下对应组件
检查规则阈值	src/core/checks/
规则库内容	knowledge/entries/*.yaml（改完要跑 pnpm sync:rules）
模型换成别的	src/embedding/loadModel.ts 的模型 id + scripts/fetch-model.mjs
大模型供应商	只改环境变量（LLM_BASE_URL / LLM_MODEL），代码不用动

最后一句：部署不是「点一下 Deploy」就完了 —— 请务必跑一遍第 7 节的 5 条验证。尤其第 4 条（语义就绪），大量「看起来部署成功」的实例其实是端侧模型没加载，只是没人点到那一页。